

Advanced Programming Course Project

Main document v0.2

A.Y. 2025/2026

This document describes the course project of the 2025/2026 A.Y.

Always adhere to these **coding principles**:

- No unsafe code.
- No undocumented panics.
- The code must be extensively tested.
- Do not preclude depending crates to test their code.
- Expose through public interfaces only what is strictly needed.
- Write idiomatic code.
- The only allowed languages are Rust (and Typescript for web frontend).

Configure your IDE or editor to use rustfmt so that the code of from all team members is consistently formatted.

Furthermore, try to minimize the warnings produced by clippy (you can also configure your IDE or editor to show clippy's warnings):

```
cargo clippy -- -W clippy::pedantic
```

Contents

1	High-level description	3
1.1	GALAXY	3
1.2	PLANETS	3
1.3	EXPLORERS	3
2	Software components	4
2.1	Sunray	4
2.2	Asteroid	4
2.3	Energy cell	4
2.4	Rocket	4
2.5	Resource	4
2.6	PLANET	5
2.6.1	Construction parameters	5
2.6.2	AI	5
2.6.3	Resource generation rule	5
2.6.4	Resource combination rule	6
2.6.5	Energy cell limit and rocket limit rules	6
2.6.6	World types	6
2.7	Explorer	7
2.7.1	Construction parameters	7
2.7.2	Bag	7
2.7.3	The EXPLORER-PLANET protocol	7
2.7.4	AI	7
2.8	Orchestrator	8

2.8.1	Initialization of the game	8
2.8.2	Logic	8
2.8.3	API	8
2.9	Galaxy visualizer	8
3	Contributions	9
3.1	Contractors	9
4	Miscellaneous	9
4.1	User-inconstructible structs	9
4.2	Distributed logging	9
4.3	Request-response between components	10
4.4	Visualization of components with channels	10

1 High-level description

In this project, you will develop a game set in a GALAXY where EXPLORERS travel through its PLANETS.

There is an interplay of several software components that you will develop either as a group or individually. Before describing the components in detail, we present a short description of the game and provide some terminology.

Later in the course we will host a software fair: at the fair, your group will buy seven PLANETS developed by other groups that you deem most interesting. You will later plug these PLANETS with your own into your game, and demonstrate the game at the exam.

1.1 Galaxy

A GALAXY is a collection of up to seven PLANETS, possibly connected to each other. Connections are bidirectional: if PLANET A is directly connected with PLANET B, then B is connected with A; that is, they are neighboring PLANETS. Connections are essentially space routes.

The topology of the GALAXY describes the connection between PLANETS. This topology does not change over time (the only exception being when a PLANET is destroyed by an asteroid, which is described later in this document) but is not known in advance by EXPLORERS (which are described later). Any topology is admissible.

A GALAXY will send sunrays towards its PLANETS, but sometimes it will send asteroids too.

1.2 Planets

A PLANET is characterized by four parameters: energy cells, rockets, resources, and combination rules; altogether, these four parameters identify the **planet type**.

Each PLANET can have one or more than one **energy cells** that store energy from **sunrays** via charging. That is, certain PLANETS types can have just one energy cell. Energy cells cannot be moved; that is, they are local to the PLANET.

Each PLANET is characterized by the **basic resource** it supports to **generate** and the **complex resources** it supports to **combine**.

Each PLANET can use energy cells to construct **rockets** to deflect **asteroids** on course of collision with it. Each PLANET can have zero or one rocket; that is, certain PLANET types cannot have rockets altogether. When the GALAXY sends an asteroid towards a PLANET, if the PLANET does not have a rocket, then it is destroyed and permanently removed from the GALAXY.

Worlds can host EXPLORERS, which are described next.

1.3 Explorers

An EXPLORER is an entity that spawns in a PLANET and travels through the GALAXY. The GALAXY must contain at least one and at most two EXPLORERS: when no EXPLORER remains, the game ends.

An EXPLORER can discover new PLANETS and interact with the PLANET hosting it in two main ways. The first interaction is used to generate the basic resource characteristic of that PLANET using energy cells. The second interaction is used to combine resources into a complex resource characteristic of the PLANET. An EXPLORER must not generate basic resources by itself.

An EXPLORER can put obtained resources in its bag, which it carries when traveling.

An EXPLORER cannot initiate interactions with other EXPLORERS.

2 Software components

The project consists of several components, each described in its own section below. Each component is developed in group or individually: see [section 3, Contributions](#). Each component has some code provided in the protocol file. Some components communicate with each other using Rust channels that are depicted in [section 4.4, Visualization of components with channels](#).

The user can only interact with the orchestrator.

2.1 Sunray

A sunray is a struct whose code is provided in the protocol file.

2.2 Asteroid

An asteroid is a struct whose code is provided in the protocol file.

2.3 Energy cell

An energy cell is a struct whose code is provided in the protocol file.

When an energy cell is constructed it has zero initial charge. It can be charged with a sunray ([section 2.1, Sunray](#)): a charge increases the charge to one. The maximum charge is one.

2.4 Rocket

A rocket is a struct whose code is provided in the protocol file.

The construction of a rocket requires one energy cell that must be fully charged.

2.5 Resource

A resource is a struct whose code is provided in the protocol file.

Resources are *basic* or *complex*. The EXPLORER can obtain the former using the *resource generation rule* ([section 2.6.3, Resource generation rule](#)) and the latter with the *resource combination rule* ([section 2.6.4, Resource combination rule](#)) which are rules for interacting with a PLANET.

The Working Group (WG) must define, at least:

- The list of basic resources.
- The list of complex resources.
- Enums for all types of resources (`enum Resource`), basic resources (`enum BasicResource`) and complex resources (`ComplexResource`). Enums are allowed to use <https://crates.io/crates/enum-as-inner> if needed.
- The rules for combining resources.

Then, it must publish the corresponding code in a common library crate that group implementations will use as a dependency.

Resources and rules need not be particularly complex. Hint: take a look at <https://littlealchemy.com/>.

Resource combination takes two resources (either basic or complex) as input and returns a complex resource as output. The two resources are taken by value (that is, they are moved) so that they cannot be used anymore.

Resource generation and combination require a number of fully charged batteries that must be decided by the Working Group (WG).

2.6 Planet

Concretely, a PLANET is a component that the orchestrator (section 2.8) runs inside a thread. Note that a PLANET is not the thread itself — it is the logic or state that happens to be executed within one.¹

A PLANET has some construction parameters (described in section 2.6.1), an internal AI (described in section 2.6.2) and follows a series of rules (described in sections 2.6.3 to 2.6.5) which ultimately define its type (described in section 2.6.6).

2.6.1 Construction parameters

A PLANET is constructed with the following parameters:

- ❶ `Option<Rocket>`
- ❷ `mpsc::Receiver<OrchestratorToPlanet>`
- ❸ `mpsc::Sender<PlanetToOrchestrator>`
- ❹ `mpsc::Receiver<ExplorerToPlanet>`
- ❺ `mpsc::Sender<PlanetToExplorer>`

A PLANET starts with an optional rocket ❶ that can be used as soon as necessary.

A PLANET uses ❷ to receive messages from the orchestrator. The list of these messages is presented in the protocol file. These messages are requests. Figure 1 depicts this approach:

- The orchestrator has a different `mpsc` sender for each PLANET. Each PLANET has a receiver which is ❷.
- The orchestrator has a single `mpsc` receiver. Each PLANET has a *copy* of the sender end which is ❸. To enable the orchestrator to tell which the PLANET it receives a message from, each message sent in ❸ must include an identifier of the PLANET that has sent it.

A PLANET uses ❹ to receive messages from EXPLORERS. The complete list of these messages and their detailed format is presented in the protocol file. Figure 3 depicts this approach:

- The PLANET has a single `mpsc` receiver which is ❹. Each EXPLORER has a *copy* of the sender end. To enable the PLANET to tell which EXPLORER it receives a message from, each message sent in the sender must include an identifier of the EXPLORER that has sent it.

2.6.2 AI

A PLANET has an internal AI that carries the following duties:

- Charges energy cells with sunrays (section 2.3, Energy cell).
- Constructs rockets with energy cells (section 2.4, Rocket).
- Deflects asteroids with rockets (section 2.2, Asteroid).

The choice of these actions is entirely up to the student who is developing the component.

2.6.3 Resource generation rule

Explorers can interact with a PLANET following the protocol described in section 2.7.3, The EXPLORER-PLANET protocol.

One such interaction is used to generate a basic resource (section 2.5, Resource). Each PLANET is characterized by the basic resource it supports to generate with energy cells: which resource is up to the group. For example, if the complete list of basic resources is air, earth, fire and water, then a PLANET can support the generation of either air, earth, fire or water. The EXPLORER can request the PLANET what basic resource it supports to generate, and the required number of fully charged energy cells, before requesting the PLANET actually generate it.

Each PLANET is also characterized by the maximum number of basic resources it supports to generate for the duration of the game: either one or unbounded.

¹When designing libraries, it is good practice not to hard-code the execution mechanism (for example, always spawning a new thread inside the library). Instead, the user of the library should be free to decide how and where to execute the code — for example, they may want to run it in their own thread, inside a thread pool, or within an async runtime.

2.6.4 Resource combination rule

Another such interaction is used to combine resources into complex resources ([section 2.5, Resource](#)) with energy cells. Each PLANET is characterized by the resource combination rule it supports: this is the rule that an EXPLORER can exercise to convert two resources into a complex resource; the decision of which rule is up to the group. For example, if the list of resource combination rules is $A + B = C$, $B + C = D$ and $A + C = E$, then a PLANET can support the combination of either $A + B$ into C , $B + C$ into D or $A + C$ into E .

The EXPLORER can request the PLANET what resource combination rule it supports, and the required number of fully charged energy cells, before requesting the PLANET actually combine a pair.

Each PLANET is also characterized by whether or not it supports a rule at all.

2.6.5 Energy cell limit and rocket limit rules

Each PLANET has a limit on the number of energy cells it can have: one or many. That is, energy cells are either stored as `EnergyCell` or `Vec<EnergyCell>`.

Each PLANET has a limit on the number of rockets it can have: zero or at most one. That is, a rocket is stored as `Option<Rocket>`.

2.6.6 World types

In light of [section 2.6.3](#), [section 2.6.4](#) and [section 2.6.5](#), we now state what PLANET types there are. There are four PLANET types. Each PLANET type specifies a value for each rule, therefore constraining the capabilities of the PLANET and those of an EXPLORER interacting with it. When starting, each group must decide the PLANET type and develop the PLANET accordingly.

PLANET type	Energy cells	Resource generation rule	Rocket	Resource combination rule
A	<code>Vec<EnergyCell></code>	at most one	at most one	no
B	<code>EnergyCell</code>	unbounded	zero	one
C	<code>EnergyCell</code>	at most one	at most one	unbounded
D	<code>Vec<EnergyCell></code>	unbounded	zero	no

2.7 Explorer

Concretely, an EXPLORER is a component that the orchestrator (section 2.8) runs inside a thread. Note that an EXPLORER is not the thread itself — it is the logic or state that happens to be executed within one; see the explanation footnote in section 2.6.

The EXPLORER has some construction parameters (described in section 2.7.1), an internal AI (described in section 2.7.4), interacts with a PLANET following an EXPLORER-PLANET protocol (described in section 2.7.3) and has a bag (described in section 2.7.2). The EXPLORER interacts with a PLANET autonomously and automatically, but it can also be controlled by the user as form of debugging.

2.7.1 Construction parameters

The EXPLORER is constructed with the following parameters:

- ❶ `mpsc::Receiver<OrchestratorToExplorer>`
- ❷ `mpsc::Sender<ExplorerToOrchestrator>`
- ❸ `mpsc::Sender<ExplorerToPlanet>`
- ❹ `mpsc::Receiver<PlanetToExplorer>`

The EXPLORER uses ❶ to receive messages from the orchestrator. The list of these messages is presented in the protocol file. These messages are requests; for example, to interact with the PLANET (section 2.7.3) and to start or stop the AI (section 2.7.4). Figure 2 depicts this approach:

- The orchestrator has a different `mpsc` sender for each EXPLORER. Each EXPLORER has a receiver which is ❶.
- The orchestrator has a single `mpsc` receiver. Each EXPLORER has a *copy* of the sender end which is ❷. To enable the orchestrator to tell which EXPLORER it receives a message from, each message sent in ❷ must include an identifier of the PLANET that has sent it.

❸ and ❹ pertain to section 2.7.3.

2.7.2 Bag

The EXPLORER starts with a bag. The bag is the set of resources (section 2.5, Resource) it has obtained by interacting with PLANETS (section 2.6, PLANET). The bag could be, for example, a `Vec<BasicResource>` and a `Vec<ComplexResource>`. Remember that the bag cannot contain energy cells, which are local to a PLANET and cannot be moved.

2.7.3 The Explorer-Planet protocol

The interaction of an EXPLORER with the PLANET it is hosted in follows a request-response pattern. Figure 3 depicts this approach:

- The PLANET has a single `mpsc` receiver. Each EXPLORER has a *copy* of the sender end which is ❸.
- The EXPLORER has a single `mpsc` receiver which is ❹. Each PLANET has a *copy* of the sender end. To enable the EXPLORER to tell which EXPLORER from which a message comes, each message sent in the sender end must include an identifier of the PLANET that has sent it.

This protocol, and the list of messages composing it, are presented in the protocol file. These are essential to the success of the project: the Working Group (WG) and contractors must improve it over time.

Remember that each PLANET has a specific PLANET type which determines the content of its responses (section 2.6.6).

2.7.4 AI

The EXPLORER has an AI that can be started with a command from the orchestrator ❶: once started, the EXPLORER starts interacting with the PLANET (section 2.6, PLANET) autonomously and automatically (that is, without user intervention). The AI can be stopped with a command.

The choice of these actions is entirely up to the student who is developing the component.

2.8 Orchestrator

The orchestrator initializes the game (section 2.8.1), has an internal logic governs the galaxy (described in section 2.8.2), and an API for directly modifying the game as a form of debugging (described in section 2.8.3). Interaction with PLANETS follows a protocol which is presented in the protocol file.

The other components of the game must not outlive the orchestrator.

2.8.1 Initialization of the game

The orchestrator initializes the game. To do so, it reads a local *Galaxy initialization file* that encodes the topology and is presented in the protocol file.

Concretely, it spawns a thread for each PLANET (section 2.6, PLANET), and provides the required parameters to it, including the necessary Rust channels.

2.8.2 Logic

The orchestrator has a logic that can be started with a command of the API (section 2.8.3): once started, the orchestrator starts the AI of EXPLORERS (section 2.7.4) and sends sunrays and asteroids to PLANETS (section 2.1, section 2.2). The logic can be stopped with a command of the API, in which case it stops the AI of the EXPLORERS and stops sending sunrays and asteroids.

2.8.3 API

The orchestrator has an API that is the interface for a user to send sunrays and asteroids to PLANETS and interacting with EXPLORERS. Essentially, the API enables the user to manually play or debug the game. Examples methods are the one for generating a resource and moving the EXPLORER to another neighboring PLANET; the latter requires the orchestrator to send the EXPLORER a message containing the sender to the new PLANET.

The methods of this API are synchronous; that is, they return whenever the intended effect has ultimately taken place.

Example usage Suppose the user wants to generate a resource, via the related method

- The orchestrator send the `GenerateResourceRequest` message to the specified EXPLORER, which receives it.
- The EXPLORER sends the `GenerateResourceRequest` message to the PLANET in which it is hosted in and obtains the result back by means of a message.
- The EXPLORER sends a confirmation to the orchestrator.

2.9 Galaxy visualizer

The GALAXY visualizer is responsible for visualizing the internal state of PLANETS and EXPLORERS. This comprehends the energy cells and resources it owns, the EXPLORER it hosts and so on. The visualizer must update whenever the states changes.

The visualizer should comprehend the orchestrator UI.

3 Contributions

Each component is either developed in group or individually. The role of each contribution is explained later in this document.

Each component must be of the specified Rust type.

Component	Contribution made by:
Orchestrator	Group
Energy cell, Rocket, Resources, Asteroid, Protocol-related structs and common code	WG & Contractors
PLANET + AI	Group (to be sold at the software fair)
EXPLORER	Individual
GALAXY visualizer	Individual

Ideally, each PLANET should be called like the group that develops it.

3.1 Contractors

There is a significant amount of common code used across many components, including protocol-related structs, enums, etc. To ensure effective progress within the timeframe of the course, the development of common code must be divided among different contractors (that is, student groups taking on additional development tasks). The subdivision of this work will be organized by the Working Group (WG). Contractors will receive additional points in the final exam for their contributions.

4 Miscellaneous

4.1 User-inconstructible structs

For correctness reasons, libraries sometimes need to make it impossible for a user to directly instantiate (that is, construct) a type. In this project, complex resources and rockets must not be user-constructible. It is still possible to cheat: an EXPLORER could construct basic resources by itself and a PLANET could create rocket by itself. This will be checked in the exam.

To make a struct user-inconstructible (often called sealed), do as follows:

Working Group (WG) library crate:

```
/// public, and directly constructible.
pub struct BasicResource;

/// public, but not constructible by dependent crates.
pub struct ComplexResource {
    /// dependent crates will not be able to name this field.
    _priv: (),
}
```

Dependent crate:

```
use ap2025::{BasicResource, ComplexResource};

fn main() {
    let br = BasicResource; // ok
    let cr = ComplexResource {
        _priv: () // error: Field '_priv' in struct 'NonConstructible' is private
    };
}
```

4.2 Distributed logging

Debugging distributed code is notoriously difficult. To ease the process, components must log incoming and outgoing messages with a crate such as log or tracing: <https://docs.rs/log/latest/log/>, <https://docs.rs/tracing/>

[latest/tracing/](#). The format of messages and event targets must be standardized by the Working Group (WG).

4.3 Request-response between components

The request-response pattern between components can be implemented using channels: the requester includes a `oneshot` sender in the request, which the intended responder can use to respond with a response.

1. The requester creates a bounded crossbeam channel: <https://docs.rs/crossbeam/latest/crossbeam/channel/fn.bounded.html>. The generic type of the channel is `Request` and the capacity is 1. The channel has two ends, a sender `(s)` and a receiver `(r)`.
2. The requester creates a request message including `(s)` in it.
3. The requester sends the request message to the intended responder.
4. The intended responder receives the request message and processes it.
5. The responder creates a response message.
6. The responder sends the response message using `(s)`.
7. The requester receives the response message from the responder using `(r)`.

An advantage of this approach is that components need not maintain persistent `mpsc` receivers for receiving responses to requests.

4.4 Visualization of components with channels

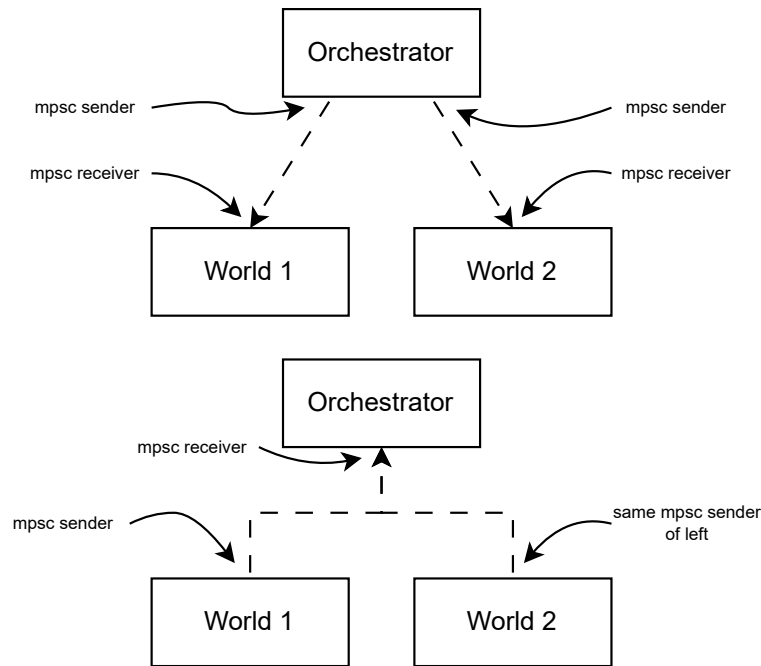


Figure 1: Channels connecting the orchestrator and the PLANET.

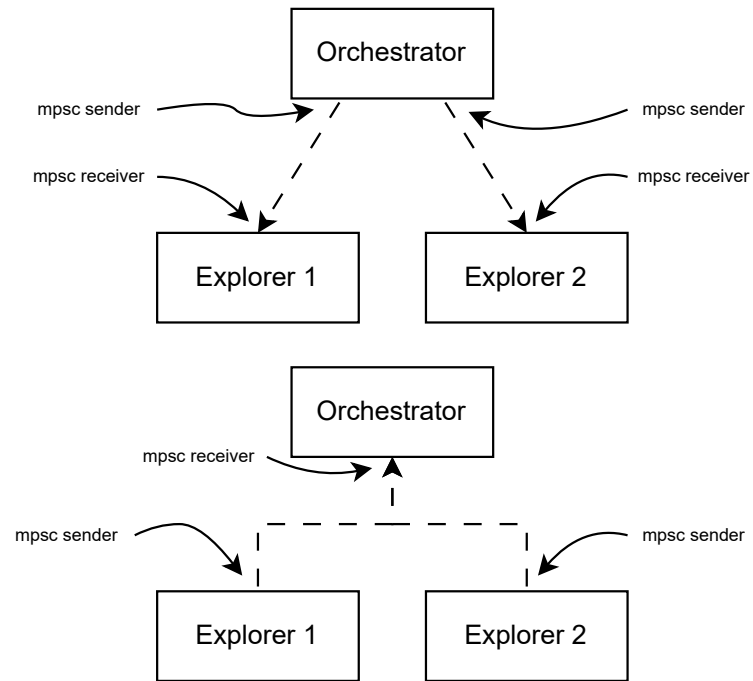


Figure 2: Channels connecting the orchestrator and the EXPLORER.

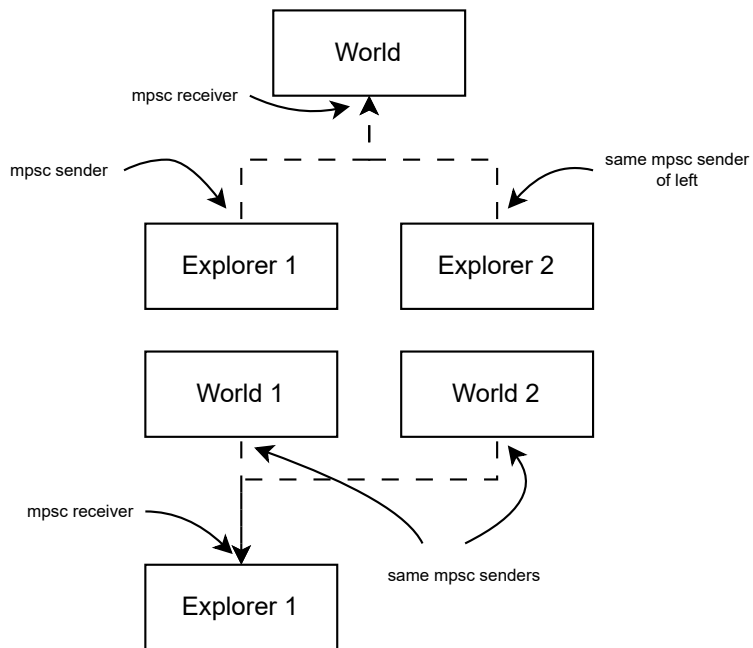


Figure 3: Channels connecting the EXPLORER and the PLANET.